
Fundamentals of Database
IT403 **Project**

Deadline: Monday 09/08/2026 @ 23:59
[Total Mark is 20]

Student Details:

CRN:50421

Name: Hatem Hassan al-Hathloul

ID: S220016911

Name: Abdullah al-Zamil

ID: S230017211

Name: Abdullah al-Zahrani

ID: S230063393

Name: Turki al-Harbi

ID: S230016935

Instructions:

- You must submit two separate copies (**one Word file and one PDF file**) using the Assignment Template on Blackboard via the allocated folder. These files **must not be in compressed format**.
- It is your responsibility to check and make sure that you have uploaded both the correct files.
- Zero mark will be given if you try to bypass the SafeAssign (e.g. misspell words, remove spaces between words, hide characters, use different character sets, **convert text into image** or languages other than English or any kind of manipulation).
- Email submission will not be accepted.
- You are advised to make your work clear and well-presented. This includes filling your information on the cover page.
- You must use this template, failing which will result in zero mark.
- You **MUST** show all your work, and text must not be converted into an image, unless specified otherwise by the question.
- Late submission will result in ZERO mark.
- The work should be your own, copying from students or other resources will result in ZERO mark.
- Use **Times New Roman** font for all your answers.

Project Instructions

- You can work on this project as a group (minimum 3 and maximum 4 students). Each group member must submit the project individually with all group member names mentioned on the cover page.
- This project **worth 20 marks** and will be distributed as in the following:
 - Database Design. (6 marks)
 - Database Implementation. (6 marks)
 - SQL Operations. (6 marks)
 - Project Reflection. (2 marks)
- Each leader in the group must submit one report about their chosen Project via the Blackboard (**Email submission will not be accepted and will be awarded ZERO marks**) containing the following:
 - A. Task 1 : database design
 - B. Task 2: database implementation
 - C. All requested queries/results
 - D. Screenshots from MySQL (or any other software you use) of **all the tables** after population and **query results**.
 - E. Reflection (300–500 words) describing the team's experience, challenges, design decisions, and AI usage (if applicable).
- Submit one **SQL file** containing the complete database implementation, including database creation, tables, data insertion, required queries, VIEW, and TRIGGER.
- Submit a **GitHub** (or GitLab) repository containing the complete project with regular commits and meaningful commit messages.
- Submit a **progress report** describing the completed work, key decisions, challenges, future plan, and contribution of each group member.
- Include **links** to the live report document, GitHub/GitLab repository, and Google Colab notebook (if applicable) in the **Project Artifacts** section of the final report.
- You are advised to make your work clear and well presented; marks may be reduced for poor presentation. This includes filling in your information on the cover page.
- You **MUST** show all your work, and text **must not** be converted into an image unless specified otherwise by the question.
- **Late submission will result in ZERO marks being awarded.**
- The work should be your own. Copying **from students or other resources, including AI software, will result in ZERO marks.**

*Learning**Outcome(s):**CLO1, CLO2, CLO3,**CLO4, CLO5*

Project Description

Design and Implementation of a Smart Clinic Database System

A private clinic currently manages its patient information manually, making it difficult to organize appointments, treatments, and payments. The clinic has decided to develop a database system to improve data organization and retrieval.

Your team is required to design, implement, and test a complete relational database using the concepts learned throughout this course. The project should demonstrate your understanding of database design, ER/EER modeling, relational mapping, SQL implementation, and advanced SQL queries. The project must be completed using MySQL.

Answer

Mid-Project Progress Report

- 1- **Work completed this period:** We successfully formed our team, defined the 7 main entities for the smart clinic system (Person, Doctor, Patient, Appointment, Treatment, Medicine, Payment), and identified their primary/foreign keys, and completed the EER diagram design including specialization.
- 2- **Key decision made:** We decide to use EER specialization approach by creating general PERSON superclass that branches into DOCTOR and PATIENT subclasses. It can reduce data redundancy for shared attribute like name and phone.
- 3- **Problem encountered:** we were confused about how to link treatments and medicines. And we resolve it by identifying it as Many-to-Many (M:N) relationship which we will later implement using bridge/junction table in SQL.

4- Plan to do next: our next step is to translate our EER diagram into MySQL code (DDL) to create table, apply the necessary constraints, and insert the mock data.

5- Group member input:

Project coordinator, writing assumptions and mid-progress report (Hatem al-Hathloul)

Defining entities, attributes, and primary/foreign keys (Turki al-Harbi)

Modeling relationships, cardinalities, and EER specialization (Abdullah al-Zahrani)

Designing and drawing the EER diagram (Abdullah al-Zamil)

Task 1 – Database Design (6 Marks)

- Design a complete database for the clinic that includes at least six entities such as Patients, Doctors, Appointments, Treatments, Medicines, Payments, or any other appropriate entities.
- A complete ER/EER diagram.
- Primary and foreign keys.
- Appropriate relationships with cardinality.
- At least one specialization/generalization (EER feature).
- Any assumptions made during the design process.

A. Entities, Attributes, and Keys**1. PERSON (Superclass)**

Attributes: Person_ID (PK), Fname, Lname, Phone.

2. DOCTOR (Subclass of PERSON)

Attributes: Person_ID (PK, FK), Specialization, Room_Number.

Note: Person_ID references PERSON.

3. PATIENT (Subclass of PERSON)

Attributes: Person_ID (PK, FK), Blood_Type, Emergency_Contact.

Note: Person_ID references PERSON

4. APPOINTMENT

Attributes: Appointment_ID (PK), App_Date, App_Time, Status, Doctor_ID (FK), Patient_ID (FK).

Note: Doctor_ID references DOCTOR(Person_ID), Patient_ID references PATIENT(Person_ID).

5. TREATMENT

Attributes: Treatment_ID (PK), Description, Cost, Appointment_ID (FK).

Note: Appointment_ID references APPOINTMENT.

6. MEDICINE

Attributes: Medicine_ID (PK), Med_Name, Dosage.

7. PRESCRIPTION (Bridge table for M:N)

Attributes: Prescription_ID (PK), Treatment_ID (FK), Medicine_ID (FK).

Note: Treatment_ID references TREATMENT, Medicine_ID references MEDICINE.

8. PAYMENT

Attributes: Payment_ID (PK), Amount, Pay_Method, Pay_Date, Appointment_ID (FK).

Note: Appointment_ID references APPOINTMENT.

B. Relationships, Cardinality, and Specialization

EER specialization/generalization: for the EER design, we opted to build a primary superclass named PERSON, which then splits into two distinct subclasses. DOCTOR and PATIENT. We set this specialization with a Total and Disjoint (d) constraint, ensuring that any given entity is strictly be either a Doctor or a Patient-never both simultaneously.

Relationship and cardinality:

Patient and appointment (1:N): we set the relationship between patient and appointment as 1:N this reflects that an individual patient is free to book multiple appointments over time, whereas any single appointment records belong strictly to one patient.

Doctor and appointment (1:N): the doctor and appointment link is structured as 1:N.

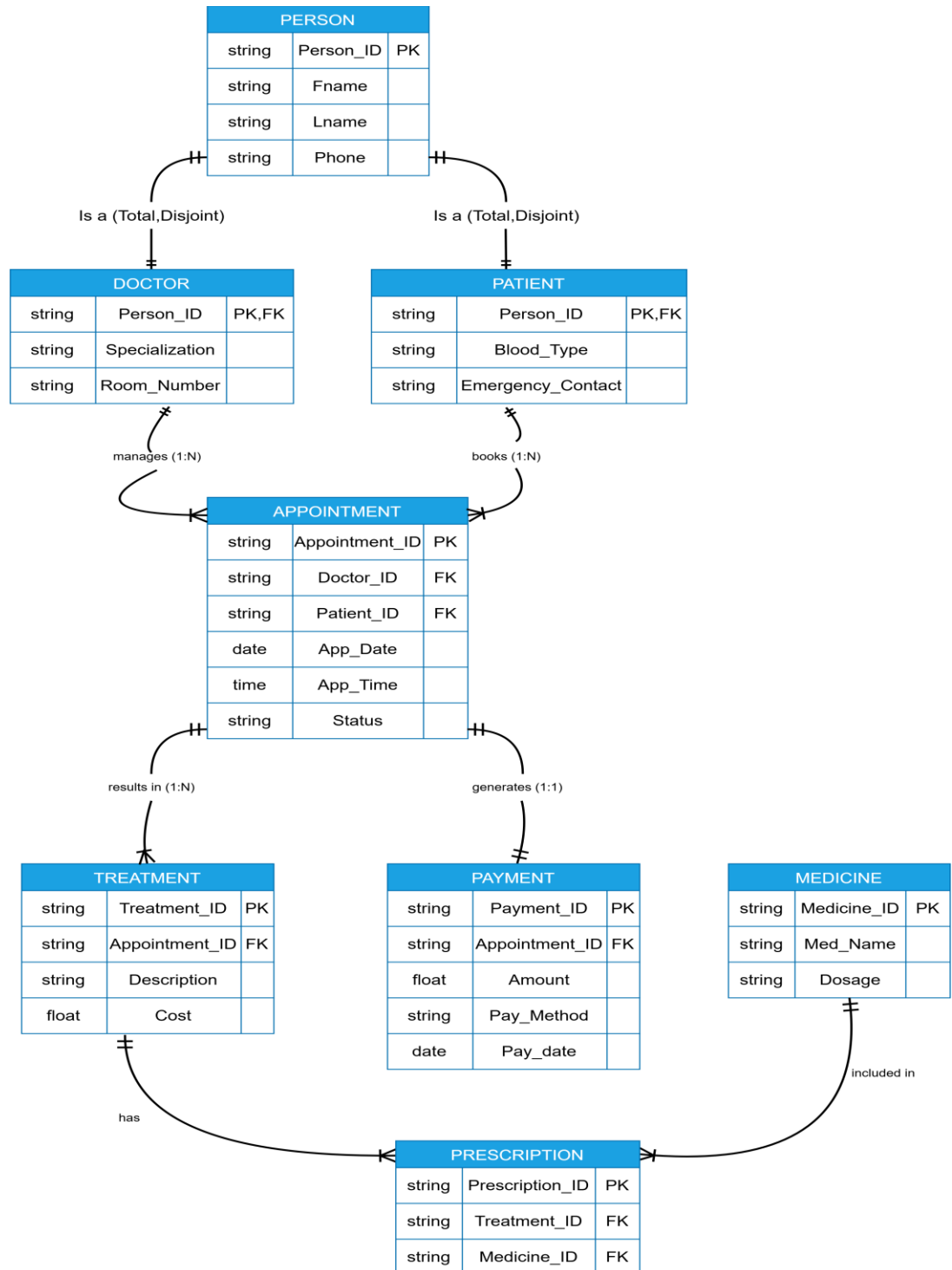
In practice, a single doctor takes on numerous appointments, but each specific appointment entry is assigned to just one doctor.

Appointment and treatment (1:N): we modeled appointment and treatment with a 1:N relationship. A single appointment session can easily involve several different treatments, but each treatment log remains tied to that specific appointment.

Appointment and payment (1:1): the mapping between appointment and payment kept as 1:1 each scheduled appointment triggers one distinct payment transaction maps back to a single appointment.

Treatment and medicine (M:N): Lastly, the relationship between treatment and medicine is defined as M:N a given treatment plan often calls for multiple medications, while a single type of medicine can naturally be used across various treatments.

C. EER Diagram



D. Assumptions

During the design of database for smart clinic, the following assumptions were made

- 1- **Total and Disjoint Specializations:** A person in the clinic system must be exclusively either a Doctor or a patient and every person recorded must belong to one of these two roles (Total Completeness).
- 2- **Payment participation (1:1):** Every scheduled appointment generates exactly one payment record (Total/Mandatory participation). In the case of a "Cancelled" appointment, a payment record is still generated but may reflect a zero amount (\$0) or a cancellation penalty fee. This ensures full data tracking for all appointments.
- 3- **M:N Relationship implementation:** To manage the Many-to-Many relationship between treatment and medicine, we implemented a Prescription bridge table and this allows a single treatment to involve multiple medicine, and a specific medicine to be prescribed across various treatment.
- 4- **EER Notation Standard:** We utilized the crow's foot notation to design our EER diagram, as it is the industry standard for relational databases. Since crow's foot does not use explicit geometric symbols (like a circle with **d**) for specialization the Total and Disjoint (d) constraint has been logically enforced in our schema design and explicitly stated in these assumptions.

E. Project Artifacts

As per the project requirements, below are the direct links to our live collaborative workspace and the version control repository. Both links have been set to Public Anyone with a link can view to ensure seamless access for evaluation.

- **Live Report Document (Collaborative Workspace):** ([Live Document](#)).
Note: Viewing this document via Word Online might cause slight pagination shifts, displacing the EER diagram or tables across pages. Please refer to the submitted PDF attachment for the exact and intended layout).
- **GitHub Repository:** ([SEU-GitHub](#))

Task 2 – Database Implementation (6 Marks)

Complete the following tasks using MySQL. Include screenshots of both your SQL code and the corresponding output (results) for each task:

- Create the database and all required tables.
- Use appropriate data types and constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, etc.).
- Insert at least 5 records into each main table.

Database Creation and Data Insertion

The complete SQL script written and committed to our repository by Abdullah ALZamil and Turki ALHarbi can be found in our GitHub repository ([Smart-Clinic-Database](#))

The screenshot displays the MySQL Workbench interface. The left sidebar shows the 'SCHEMAS' panel with a tree view containing 'mytestngdb', 'smart_clinic', 'Tables', 'Views', 'Stored Procedures', and 'Functions'. The main editor window shows a SQL script with the following content:

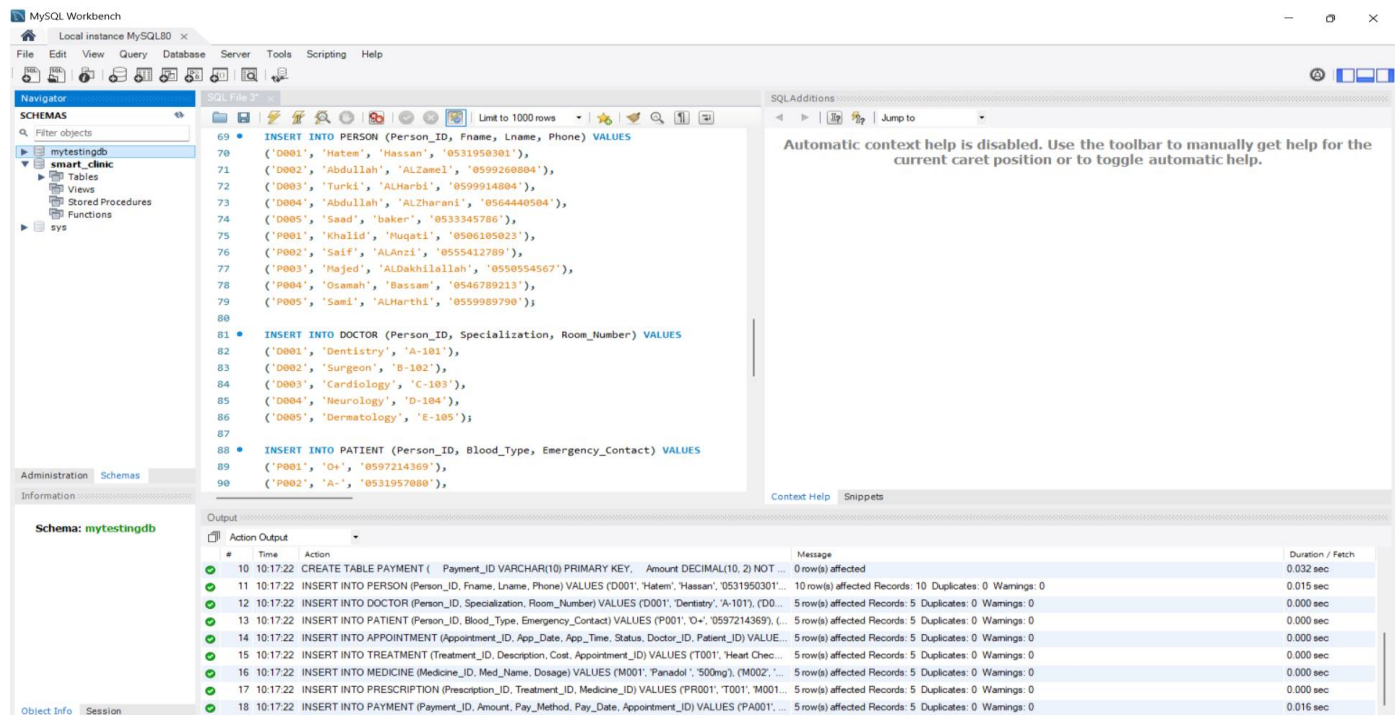
```

69 • INSERT INTO PERSON (Person_ID, Fname, Lname, Phone) VALUES
70 ('D001', 'Hatem', 'Hassan', '0531950301'),
71 ('D002', 'Abdullah', 'ALZamel', '0599260804'),
72 ('D003', 'Turki', 'ALHarbi', '0599914804'),
73 ('D004', 'Abdullah', 'ALZharani', '0564440504'),
74 ('D005', 'Saad', 'Baker', '0533345786'),
75 ('P001', 'Khalid', 'Hugati', '0506105023'),
76 ('P002', 'Saif', 'ALAnzi', '0555412789'),
77 ('P003', 'Majed', 'ALDakhilallah', '0505554567'),
78 ('P004', 'Osamah', 'Bassam', '0546789213'),
79 ('P005', 'Sami', 'ALHarthi', '0559989790');
80
81 • INSERT INTO DOCTOR (Person_ID, Specialization, Room_Number) VALUES
82 ('D001', 'Dentistry', 'A-101'),
83 ('D002', 'Surgeon', 'B-102'),
84 ('D003', 'Cardiology', 'C-103'),
85 ('D004', 'Neurology', 'D-104'),
86 ('D005', 'Dermatology', 'E-105');
87
88 • INSERT INTO PATIENT (Person_ID, Blood_Type, Emergency_Contact) VALUES
89 ('P001', 'O+', '0597214369'),
90 ('P002', 'A-', '0531957080');

```

The right sidebar shows the 'SQL Additions' panel with a message: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help." The bottom panel shows the 'Output' window with a table of execution results:

#	Time	Action	Message	Duration / Fetch
10	10:17:22	CREATE TABLE PAYMENT (Payment_ID VARCHAR(10) PRIMARY KEY, Amount DECIMAL(10, 2) NOT ...	0 row(s) affected	0.032 sec
11	10:17:22	INSERT INTO PERSON (Person_ID, Fname, Lname, Phone) VALUES (D001, 'Hatem', 'Hassan', '0531950301'...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec
12	10:17:22	INSERT INTO DOCTOR (Person_ID, Specialization, Room_Number) VALUES (D001, 'Dentistry', 'A-101'), (D0...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
13	10:17:22	INSERT INTO PATIENT (Person_ID, Blood_Type, Emergency_Contact) VALUES (P001, 'O+', '0597214369'), (...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
14	10:17:22	INSERT INTO APPOINTMENT (Appointment_ID, App_Date, App_Time, Status, Doctor_ID, Patient_ID) VALUE...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
15	10:17:22	INSERT INTO TREATMENT (Treatment_ID, Description, Cost, Appointment_ID) VALUES (T001, 'Heart Chec...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
16	10:17:22	INSERT INTO MEDICINE (Medicine_ID, Med_Name, Dosage) VALUES (M001, 'Panadol', '500mg'), (M002, '...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
17	10:17:22	INSERT INTO PRESCRIPTION (Prescription_ID, Treatment_ID, Medicine_ID) VALUES (PR001, 'T001', 'M001'...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
18	10:17:22	INSERT INTO PAYMENT (Payment_ID, Amount, Pay_Method, Pay_Date, Appointment_ID) VALUES (PA001, ...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.016 sec



We successfully created 8 tables with appropriate Primary and Foreign keys based on our EER diagram. We then populated each table with 5 mock records. The execution was successful without errors, as shown in the screenshot above.

Task 3 – SQL Operations (6 Marks)

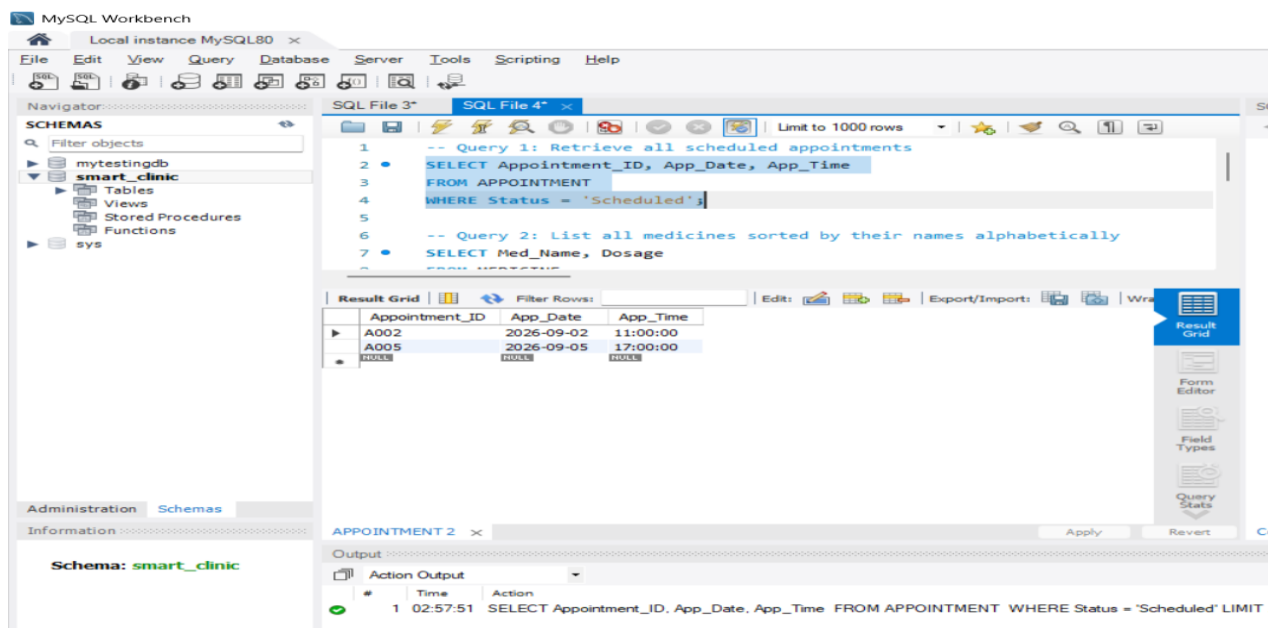
Complete the following tasks using MySQL. For each task, include screenshots of both your SQL code and the corresponding output (results), along with a brief description (one to two sentences) explaining the purpose of the query and the information it is intended to retrieve.

- Write SELECT statements.
- Write JOIN queries.
- Write nested queries.
- Use aggregate functions with GROUP BY.
- Write UPDATE and DELETE statements.
- Create one VIEW.
- Create one TRIGGER.

The SQL queries for Task 3 have been successfully written and committed to our repository by Abdullah Al-Zahrani. The dedicated query file can be accessed here: ([clinic_queries.sql](#))

Query 1: This query retrieves the IDs, dates, and times of all appointments that are currently marked as Scheduled:

```
SELECT Appointment_ID, App_Date, App_Time
FROM APPOINTMENT
WHERE Status = 'Scheduled';
```



Query 2: This query lists all available medicines in the clinic, ordered alphabetically by their names for easier tracking:

```
SELECT Med_Name, Dosage
FROM MEDICINE
ORDER BY Med_Name ASC;
```

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'smart_clinic' selected. The main editor window shows a SQL query in 'SQL File 4*'. The query is as follows:

```
4 WHERE Status = 'Scheduled';
5
6 -- Query 2: List all medicines sorted by their names alphabetically
7 SELECT Med_Name, Dosage
8 FROM MEDICINE
9 ORDER BY Med_Name ASC;
10
```

Below the query editor, the 'Result Grid' is displayed, showing the results of the query:

Med_Name	Dosage
Ibuprofen	300mg
Lisinopril	40mg
Loratadine	30mg
Panadol	500mg
Vitamin C	500mg

The bottom panel shows the 'Output' tab with 'Action Output' selected, displaying the execution log:

#	Time	Action
1	02:57:51	SELECT Appointment_ID, App_Date, App_Time FROM APPOINTMENT WHERE Status = 'Scheduled' LIM
2	03:00:32	SELECT Med_Name, Dosage FROM MEDICINE ORDER BY Med_Name ASC LIMIT 0, 1000

Query 3: This query identifies treatments that cost more than 200, which helps in filtering high-cost medical procedures:

```
SELECT Description, Cost
FROM TREATMENT
WHERE Cost > 200.00;
```

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'smart_clinic' selected. The main editor window shows a SQL query file with the following code:

```
8 FROM MEDICINE
9 ORDER BY Med_Name ASC;
10
11 -- Query 3: Find treatments that cost more than 200
12 • SELECT Description, Cost
13 FROM TREATMENT
14 WHERE Cost > 200.00;
```

Below the query editor, the 'Result Grid' is displayed, showing the results of the query:

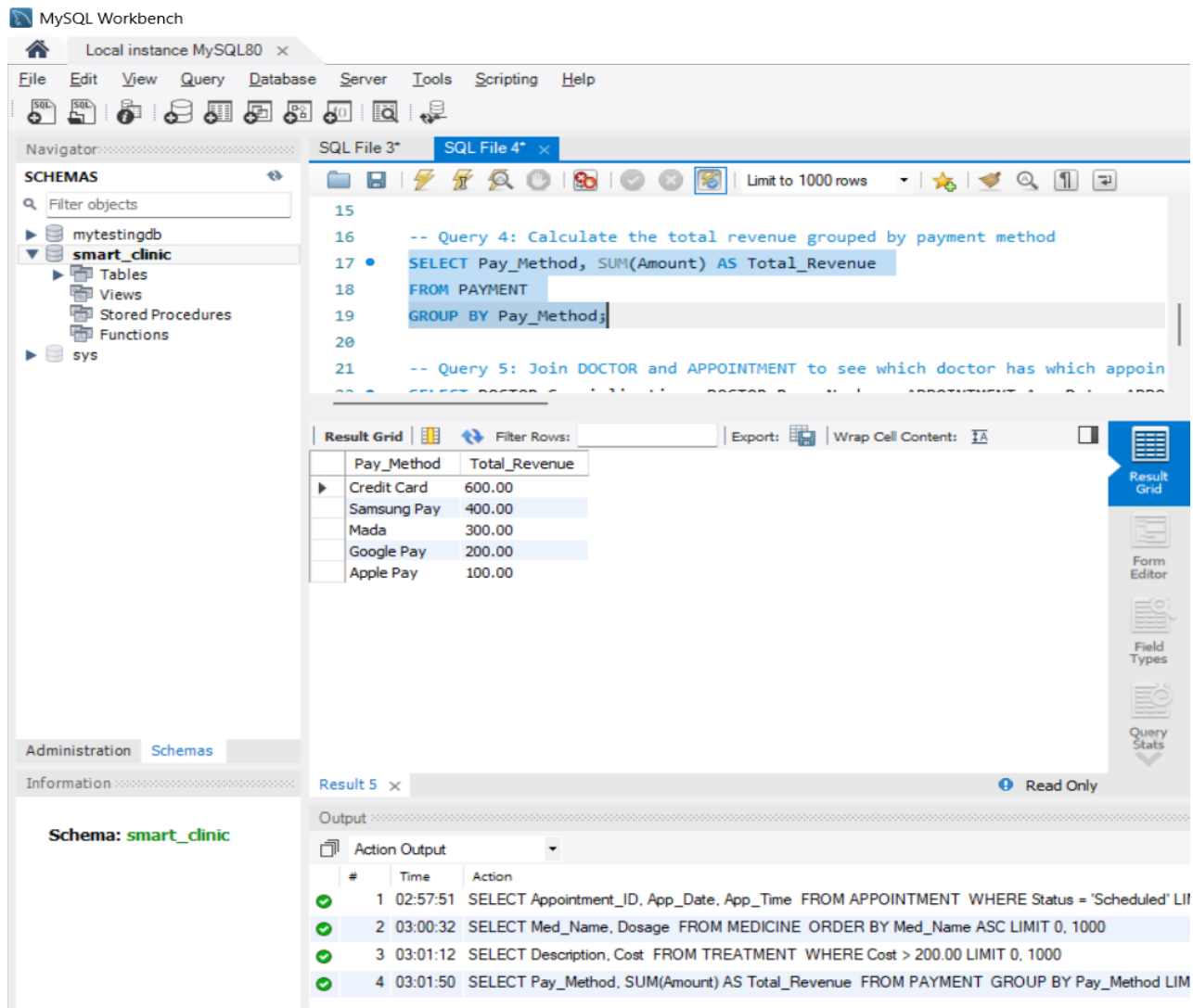
Description	Cost
Heart Checkup	600.00
Skin Allergy Test	400.00
Blood Pressure Test	300.00

The bottom section of the interface shows the 'Output' tab with 'Action Output' selected, displaying a log of executed queries:

#	Time	Action
1	02:57:51	SELECT Appointment_ID, App_Date, App_Time FROM APPOINTMENT WHERE Status = 'Scheduled' LIMIT 0, 1000
2	03:00:32	SELECT Med_Name, Dosage FROM MEDICINE ORDER BY Med_Name ASC LIMIT 0, 1000
3	03:01:12	SELECT Description, Cost FROM TREATMENT WHERE Cost > 200.00 LIMIT 0, 1000

Query 4: This query calculates the total revenue generated from each payment method (e.g., Cash, Credit Card) using the aggregate function SUM():

```
SELECT Pay_Method, SUM(Amount) AS Total_Revenue
FROM PAYMENT
GROUP BY Pay_Method;
```



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' panel with a tree view showing 'mytestingdb' and 'smart_clinic'. The 'smart_clinic' schema is selected, showing 'Tables', 'Views', 'Stored Procedures', and 'Functions'. The main editor window shows the SQL query for Query 4. The 'Result Grid' tab is active, displaying the results of the query. The results are as follows:

Pay_Method	Total_Revenue
Credit Card	600.00
Samsung Pay	400.00
Mada	300.00
Google Pay	200.00
Apple Pay	100.00

The bottom panel shows the 'Output' tab with 'Action Output' selected, displaying a log of executed queries and their execution times.

Query 5: This query uses an INNER JOIN to link the DOCTOR and APPOINTMENT tables, displaying the specialization of the doctor alongside their appointment details:

```
SELECT DOCTOR.Specialization, DOCTOR.Room_Number, APPOINTMENT.App_Date,
APPOINTMENT.Status
FROM DOCTOR
JOIN APPOINTMENT ON DOCTOR.Person_ID = APPOINTMENT.Doctor_ID;
```

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

Filter objects

mytestingdb

smart_clinic

Tables

Views

Stored Procedures

Functions

sys

SQL File 3*

SQL File 4*

Limit to 1000 rows

19 GROUP BY Pay_Method;

20

21 -- Query 5: Join DOCTOR and APPOINTMENT to see which doctor has which appoin

22 • SELECT DOCTOR.Specialization, DOCTOR.Room_Number, APPOINTMENT.App_Date, APPO

23 FROM DOCTOR

24 JOIN APPOINTMENT ON DOCTOR.Person_ID = APPOINTMENT.Doctor_ID;

25

Result Grid

	Specialization	Room_Number	App_Date	Status
▶	Dentistry	A-101	2026-09-01	Completed
	Dentistry	A-101	2026-09-03	Completed
	Surgeon	B-102	2026-09-02	Scheduled
	Cardiology	C-103	2026-09-04	Cancelled
	Neurology	D-104	2026-09-05	Scheduled

Administration Schemas

Information

Schema: smart_clinic

Result 6

Read Only

Output

Action Output

#	Time	Action
✓ 1	02:57:51	SELECT Appointment_ID, App_Date, App_Time FROM APPOINTMENT WHERE Status = 'Scheduled' LIM
✓ 2	03:00:32	SELECT Med_Name, Dosage FROM MEDICINE ORDER BY Med_Name ASC LIMIT 0, 1000
✓ 3	03:01:12	SELECT Description, Cost FROM TREATMENT WHERE Cost > 200.00 LIMIT 0, 1000
✓ 4	03:01:50	SELECT Pay_Method, SUM(Amount) AS Total_Revenue FROM PAYMENT GROUP BY Pay_Method LIM
✓ 5	03:02:18	SELECT DOCTOR.Specialization, DOCTOR.Room_Number, APPOINTMENT.App_Date, APPOINTMENT.S

Query 6: A nested query to retrieve the first and last names of patients who have scheduled appointments.

```
SELECT Fname, Lname  
  
FROM PERSON  
  
WHERE Person_ID IN (  
  
    SELECT Patient_ID  
  
    FROM APPOINTMENT  
  
    WHERE Status = 'Scheduled'  
  
);
```

MySQL Workbench interface showing the execution of Query 6. The query is a nested SELECT statement to find patients with scheduled appointments. The result grid shows two rows: Saif ALAnzi and Sami ALHarthi. The bottom status bar indicates 2 row(s) returned.

```
-- Query 6: Nested Query to find patients with scheduled appointments  
SELECT Fname, Lname  
FROM PERSON  
WHERE Person_ID IN (  
    SELECT Patient_ID  
    FROM APPOINTMENT  
    WHERE Status = 'Scheduled'  
);
```

Fname	Lname
Saif	ALAnzi
Sami	ALHarthi

PERSON 5 x Read Only

Output

Action Output

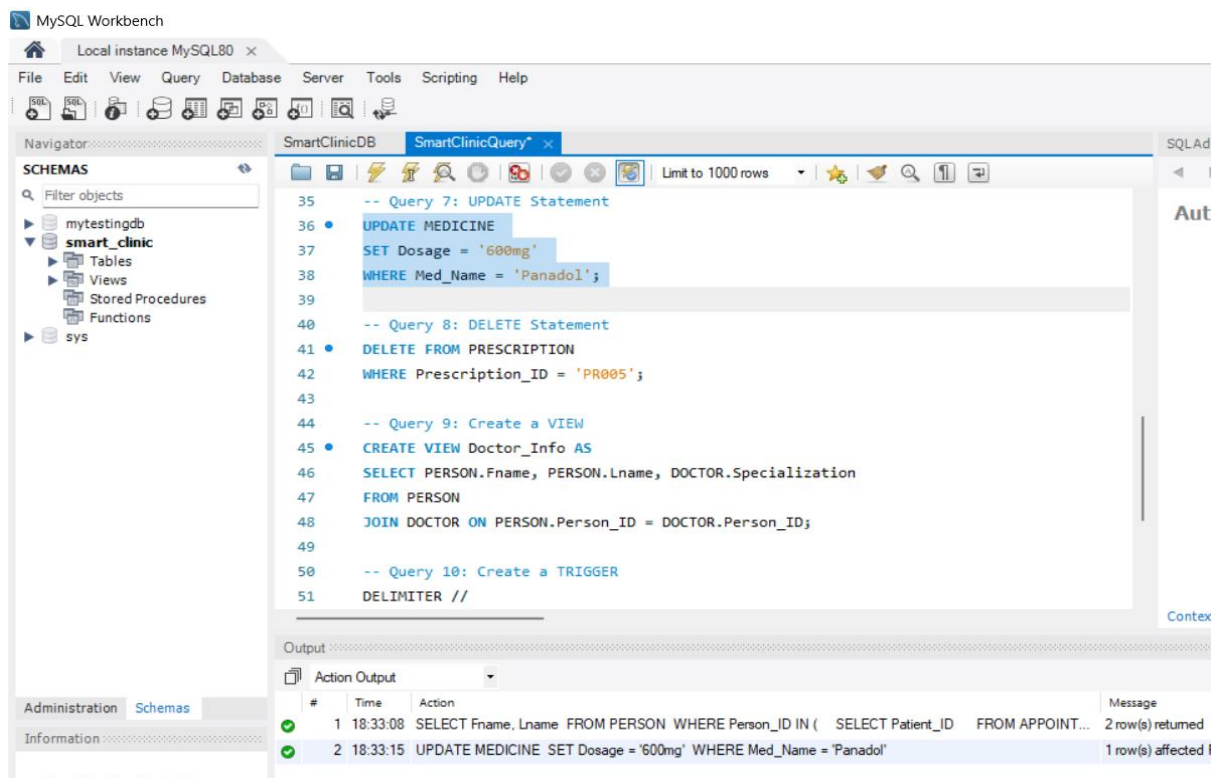
#	Time	Action	Message
1	17:08:39	SELECT Fname, Lname FROM PERSON WHERE Person_ID IN (SELECT Patient_ID FROM APPOINT...	2 row(s)

Query 7: An UPDATE statement to modify the dosage of Panadol to 600mg in the MEDICINE table.

UPDATE MEDICINE

SET Dosage = '600mg'

WHERE Med_Name = 'Panadol';



Query 8: A DELETE statement to safely remove a specific prescription (PR005) from the PRESCRIPTION table.

DELETE FROM PRESCRIPTION

WHERE Prescription_ID = 'PR005';

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'mytestingdb', 'smart_clinic', and 'sys' databases. The 'smart_clinic' database is selected, showing its tables, views, stored procedures, and functions. The main editor window shows a SQL script with three queries: Query 8 (DELETE FROM PRESCRIPTION WHERE Prescription_ID = 'PR004'), Query 9 (CREATE VIEW Doctor_Info AS SELECT PERSON.Fname, PERSON.Lname, DOCTOR.Specialization FROM PERSON JOIN DOCTOR ON PERSON.Person_ID = DOCTOR.Person_ID;), and Query 10 (CREATE TRIGGER Prevent_Negative_Cost BEFORE INSERT ON TREATMENT FOR EACH ROW BEGIN). The 'Output' tab at the bottom shows the execution results of these queries. The first query (DELETE) successfully deleted 1 row(s) from the PRESCRIPTION table.

```
39
40 -- Query 8: DELETE Statement
41 • DELETE FROM PRESCRIPTION
42   WHERE Prescription_ID = 'PR004';
43
44 -- Query 9: Create a VIEW
45 • CREATE VIEW Doctor_Info AS
46   SELECT PERSON.Fname, PERSON.Lname, DOCTOR.Specialization
47   FROM PERSON
48   JOIN DOCTOR ON PERSON.Person_ID = DOCTOR.Person_ID;
49
50 -- Query 10: Create a TRIGGER
51 DELIMITER //
52 • CREATE TRIGGER Prevent_Negative_Cost
53   BEFORE INSERT ON TREATMENT
54   FOR EACH ROW
55   BEGIN
```

Output:

#	Time	Action	Message
1	18:43:57	SELECT Fname, Lname FROM PERSON WHERE Person_ID IN (SELECT Patient_ID FROM APPOINTMENT)	2 row(s) affected
2	18:44:05	UPDATE MEDICINE SET Dosage = '750mg' WHERE Med_Name = 'Panadol'	1 row(s) affected
3	18:44:18	DELETE FROM PRESCRIPTION WHERE Prescription_ID = 'PR004'	1 row(s) affected

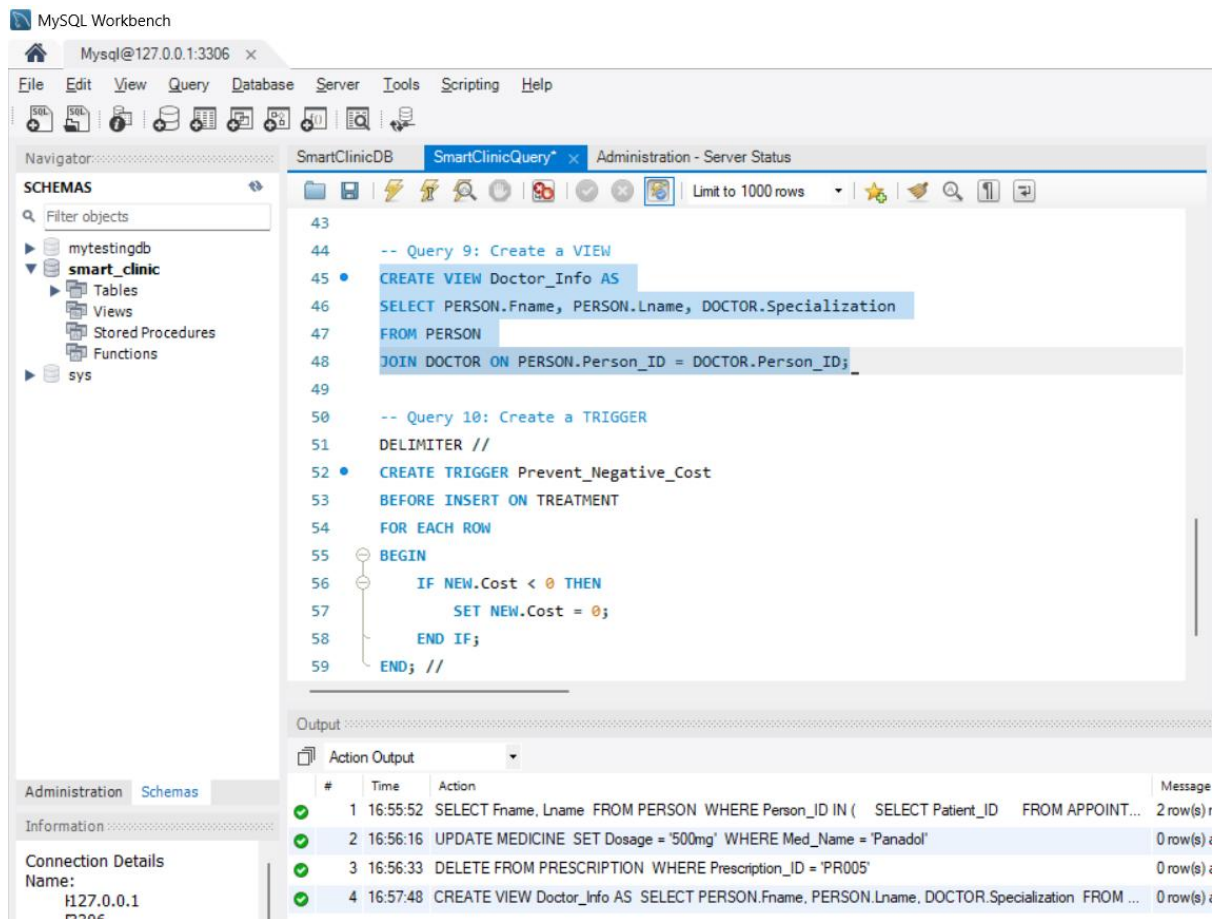
Query 9: A VIEW to display the doctor's first name, last name, and specialization by joining the PERSON and DOCTOR tables.

```
CREATE VIEW Doctor_Info AS
```

```
SELECT PERSON.Fname, PERSON.Lname, DOCTOR.Specialization
```

```
FROM PERSON
```

```
JOIN DOCTOR ON PERSON.Person_ID = DOCTOR.Person_ID;
```



Query 10: Create a TRIGGER

```
DELIMITER //
```

```
CREATE TRIGGER Prevent_Negative_Cost
```

```
BEFORE INSERT ON TREATMENT
```

```
FOR EACH ROW
```

```
BEGIN
```

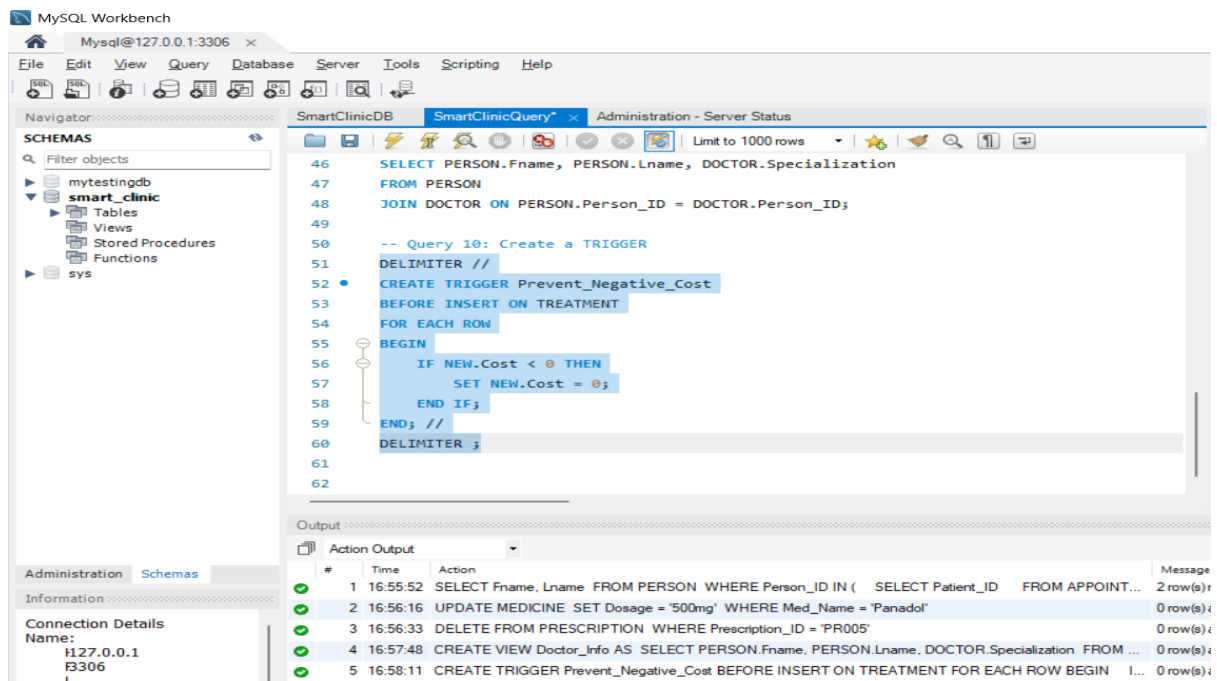
```
IF NEW.Cost < 0 THEN
```

```
SET NEW.Cost = 0;
```

```
END IF;
```

```
END; //
```

```
DELIMITER ;
```



Task 4 – Reflection (2 Marks)

Write a **300–500-word reflection** on your experience while completing this project by answering the following questions. Your responses should be clear, concise, and supported with specific examples from your own work.

1. Describe one specific challenge or obstacle you encountered during this project and explain in detail how you identified and resolved it.
2. Explain why you chose your specific approach/method/model over at least one alternative you considered, and what tradeoffs informed that decision.
3. If you used any AI tools during this project (e.g., ChatGPT, Claude, Copilot), disclose which tool(s), for what specific purpose (e.g., debugging, grammar checking, brainstorming), and how you verified or modified the output.
4. What would you do differently if you were to redo this project with more time or resources?

While we working on the Smart Clinic database project it has been a highly rewarding experience for our team. It's very beneficial and provided us the opportunity to apply theoretical database concepts—such as EER diagram design, schema creation, and writing SQL queries—into a practical, real-world scenario.

On the technical side, translating our initial requirements into a relational schema required careful planning. We had to accurately define Primary and Foreign keys to ensure data integrity, especially when establishing the relationships between the DOCTOR, PATIENT, and APPOINTMENT tables.

We chose to normalize the database and separate tables (such as specialization and entity inheritance) rather than using a single flattened table to avoid data redundancy and maintain data integrity. While a single-table approach might simplify initial queries, it introduces severe update anomalies and data duplication. Therefore, we traded a minor increase in query complexity (using JOINS) for long-term relational integrity and scalability.

additionally, selecting the appropriate data types—such as using VARCHAR for phone numbers to preserve leading zeros—taught us the importance of attention to detail in database design.

During our final review, we noticed a cardinality notation error in our initial EER diagram between the APPOINTMENT and PAYMENT entities. We had mistakenly placed a "crow's foot" (indicating a one to Many relationship) on the PAYMENT side, which contradicted our business rule of a strictly One-to-One (1:1) relationship. We proactively corrected the diagram to replace the crow's foot with a 1:1 notation, ensuring perfect consistency between our conceptual design and our final SQL implementation.

In terms of teamwork and version control, we encountered a real collaborative challenge using GitHub. During the implementation phase, our team member, Abdullah Al-Zahrani, accidentally committed the SQL queries into the same file used for table creation by Turki and Al-Zamil.

Instead of deleting the repository history, we utilized GitHub's capabilities to solve the problem professionally by editing the initial file to remove the misplaced queries, committing the fix with a clear message, and then creating a dedicated clinic_queries.sql file.

Overall, this project significantly improved our technical SQL programming skills and our ability to collaborate effectively and resolve conflicts as a software development team.

Looking forward, if we were to expand this project, we would consider developing a simple web-based user interface (UI) to interact with the database more seamlessly. Additionally, implementing advanced features like database triggers to automatically update appointment statuses would further optimize the clinic's workflow.

AI Usage Disclosure

In accordance with academic integrity guidelines, our team utilized an Artificial Intelligence assistant during the development of this project as a supplementary learning tool and technical guide. The AI was specifically used for the following purposes:

- **Conceptual Clarification:** To deepen our understanding of advanced database design concepts, such as clarifying the differences between Completeness (Total/Partial) and Disjointness (Disjoint/Overlapping) constraints in EER diagrams.

- **Code Comprehension and Debugging:** To break down and explain the logical execution of complex SQL commands (such as JOIN and GROUP BY), and to troubleshoot minor environmental errors within MySQL Workbench (e.g., active schema selection issues).
- **Proofreading and Structuring:** To refine the English phrasing, grammar, and structural flow of our project reflection (Task 4), ensuring our ideas were communicated clearly and professionally.

It is important to note that all core project artifacts—including the initial database design, business rules, EER diagram creation, and the logical formulation of the queries—were developed independently by the team members.

Deliverables

1. **Project Report (PDF)** – System description, ER/EER diagram, relational schema, SQL code, screenshots, and reflection.
2. **SQL Script (.sql)** – Complete SQL implementation of the project.
3. **GitHub Repository** – Repository link with the complete project and commit history.
4. **Mid-Project Progress Report** – Summary of progress, challenges, decisions, and team contributions.
5. **Project Artifacts**
 - Live report document (with edit history): [[Live Document](#)]
 - GitHub/GitLab repository (with commit history): [[GitHub](#)]
 - Google Colab notebook (if applicable): [link]